

universidade de aveiro



deti

departamento de eletrónica,
telecomunicações e informática

Complements of Machine Learning

2025/2026



What did we saw in the previous class?

- Paradigm shift:
 - Moving from from **RNNs** (sequential processing) to **Transformers** (parallel processing).
- Transformer Engine:
 - **Multi-head self-attention**: Learning different types of relationships in parallel.
 - **Positional encodings**: Giving the model a sense of order (since it has no recurrence).
- Efficient Fine-tuning (PEFT):
 - **LoRA (Low-Rank Adaptation)**: Reducing trainable parameters by 10,000x using rank decomposition,

Today's Agenda

Topic
1. Revise some core concepts
2. NLP Task with Transformers
3. (L)LM → Assistant Model

1. Transformers Recap

RNNs vs Transformers

	RNNs	Transformers
Training		
Inference		

RNNs vs Transformers

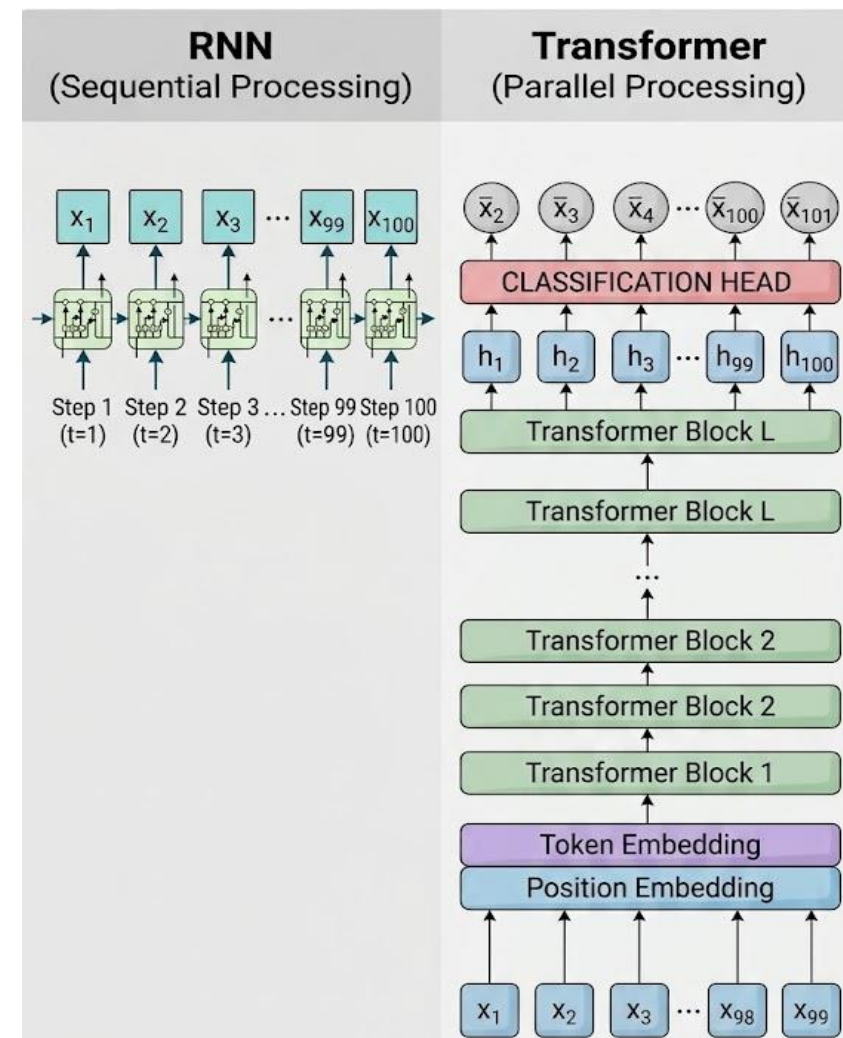
	RNNs	Transformers
Training	Slow (sequential)	Fast (parallel)
Inference	Fast (relative)	Slow(relative)

RNNs vs Transformers

- Let consider a sentence with 100 tokens.
 - How many iterations does the RNN needs to do over this sequence vs Transformer?

RNNs vs Transformers

- Let consider a sentence with 100 tokens.
 - How many iterations does the RNN needs to do over this sequence vs Transformer?



2. NLP tasks (with transformers)

Text Generation

➤ How can we generate text from a probabilistic model?

- By sampling

$$\hat{y} = \arg \max_{\{w_0, w_1, \dots, w_m\}} P(w_0, w_1, \dots, w_m | c) = \arg \max_{\{w_0, w_1, \dots, w_m\}} \prod_{i=0}^m P(w_i | w_0, w_1, \dots, w_{i-1}, c)$$

➤ Greedy search

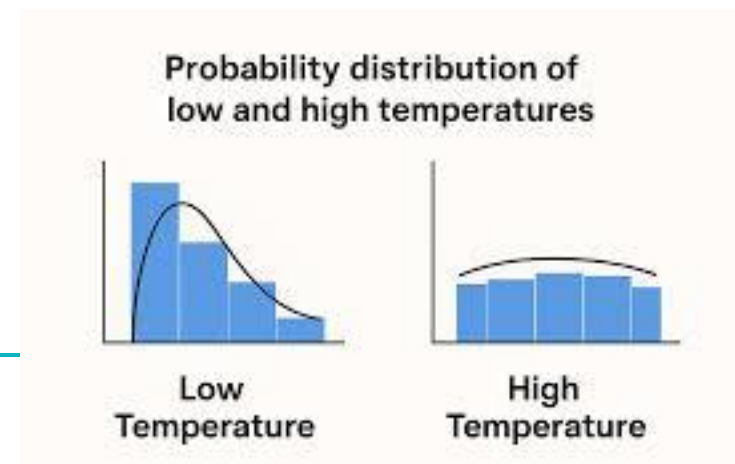
- Always selects the single most probable token.

➤ Beam Search

- Selects the most probable chain of tokens within a “beam” width.

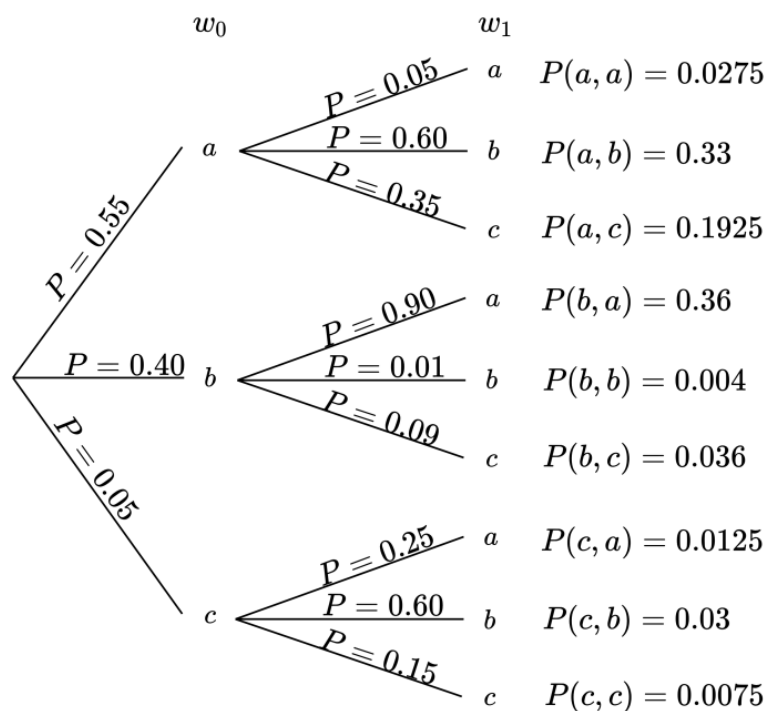
➤ Random Sampling

- Randomly samples the next token based on the probability distribution.

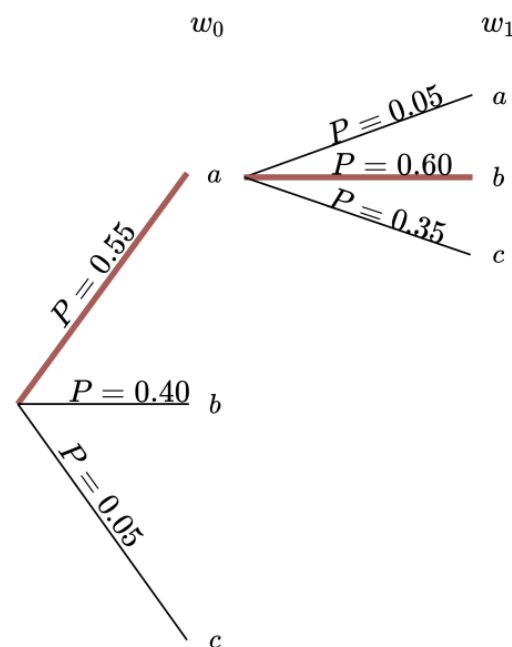


Text Generation

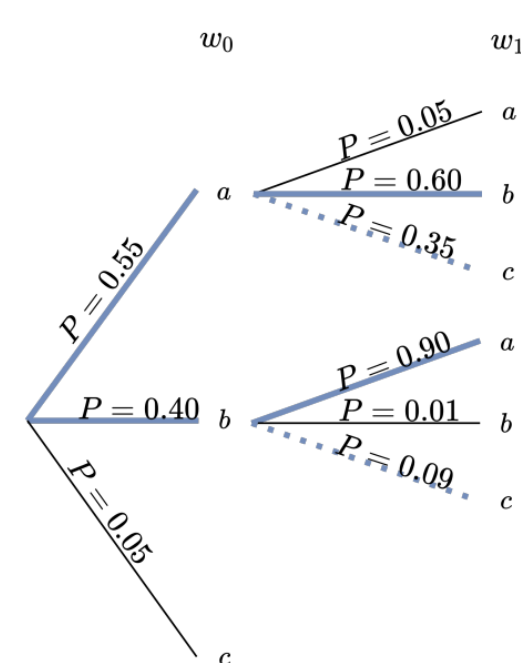
Brute Force



Greedy Search



Beam Search ($k = 2$)



Text Generation: Problems

➤ Greedy Decoding

- Text can become repetitive, generic, or get stuck in a "local optimum".
- No variance

➤ Beam Search

- Gets high quality sentence (most more probable)
- No variance
- Computationally expensive

➤ Random Sampling

- Can generate nonsensical or factually incorrect text
- High variance (hard to get two identical sentences).

Text Generation: Problems

➤ Greedy Decoding

- Text can become repetitive, generic, or get stuck in a "local optimum".
- No variance

➤ Beam Search

- Gets high quality sentence (most more probable)
- No variance
- Computationally expensive

➤ Random Sampling

- Can generate nonsensical or factually incorrect text
- High variance (hard to get two identical sentences).



Text Generation: Variants

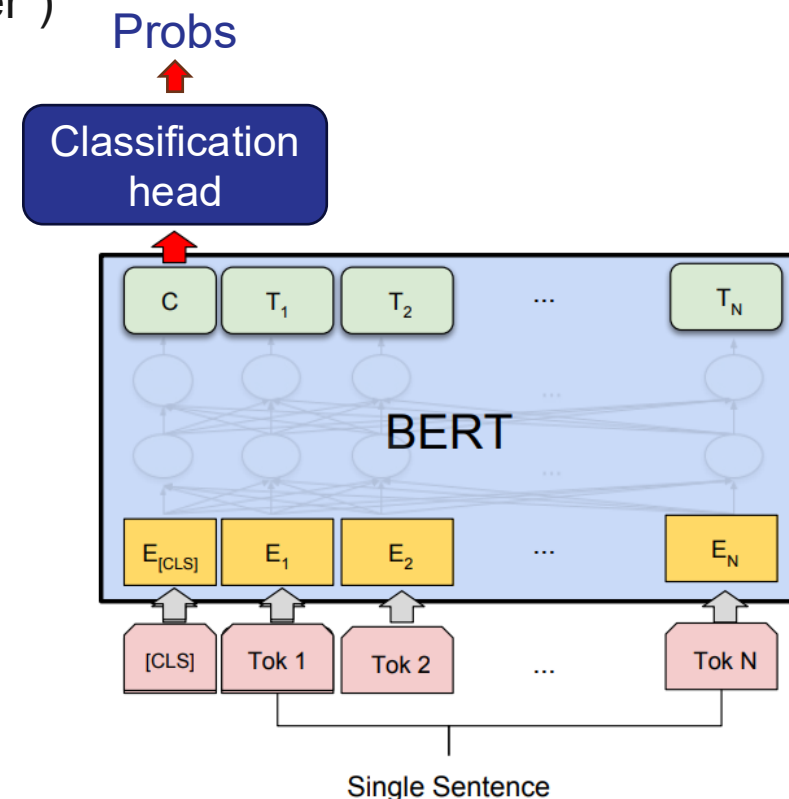
Widely
adopted

- Top-k Sampling:
 - Only considers the top k most probable tokens as valid tokens.
- Top-p (Nucleus) Sampling
 - Dynamically choose the number of tokens to sample from based on the cumulative prob.
 - More robust than top-k, adapting to spiky vs. flat distributions at each step.
- Contrastive Search
 - Aims to generate high-quality text while explicitly preventing repetition (degeneration).
 - Evaluates candidate tokens from the top-k vocabulary by balancing two competing factors:
 - **Model Confidence:** Promotes tokens with a high predicted probability given the previous context.
 - **Degeneration Penalty:** Penalizes candidate tokens whose hidden representations are too similar to previously generated tokens.

$$x_t = \arg \max_{v \in V^{(k)}} \left\{ (1 - \alpha) \times \underbrace{p_{\theta}(v | \mathbf{x}_{<t})}_{\text{model confidence}} - \alpha \times \underbrace{(\max\{s(h_v, h_{x_j}) : 1 \leq j \leq t-1\})}_{\text{degeneration penalty}} \right\},$$

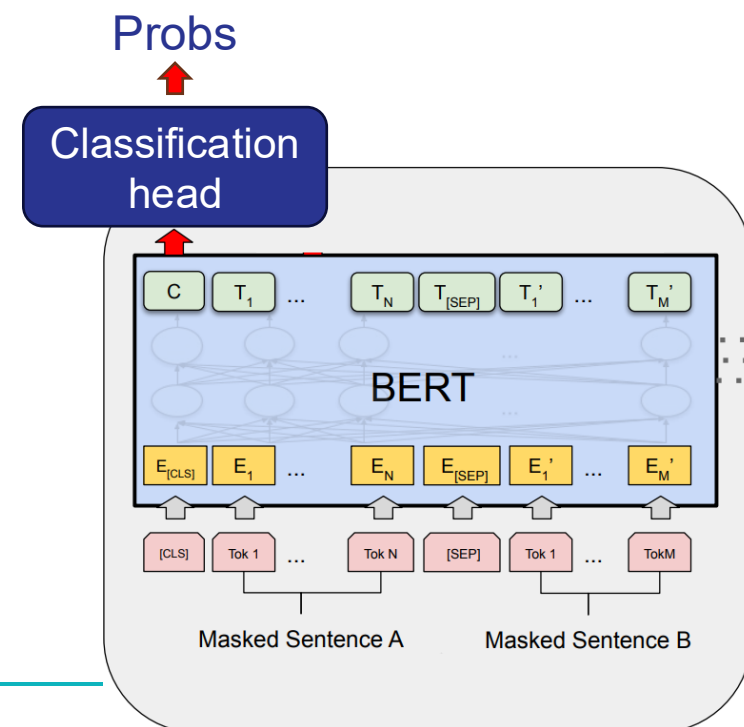
Text Classification

- Classifies the provided piece of text:
 - **Sentiment Analysis** (Positive/Negative/Neutral)
 - **Topic Categorization** (Sports/Politics/Tech)
 - **Intent Detection** (e.g., in chatbots: "Book a flight", "Cancel order")
 - **Spam/Toxicity Detection** (Spam/Not Spam)
- Framed as a classification task
- Architecture: Encoder-only like BERT



Relevance Estimation

- Estimate the relevance between two pieces of text
 - Document Reranking
 - Answer Extraction
 - Sentence Similarity
- Framed as a binary classification task
- Architecture: Encoder-only like BERT



Named Entity Recognition (NER)

- **The Goal:** Identify and categorize "entities" in unstructured text (e.g., People, Places, Organizations)

How would you solve this task?

Barack Obama Person the 44th President of the United States Title, was born in Honolulu, Hawaii Location. He graduated from Columbia University Org and Harvard Law School Org. In 2009 Date, Obama was elected as the first African American Ethnicity President of the United States Location. During his presidency, Obama implemented the Affordable Care Act Law and strengthened diplomatic relations with Cuba Location. He served two terms in office before being succeeded by President Donald Trump Title in 2017 Date.

Named Entity Recognition (NER)

➤ Naive Approach: Independent Token Classification

- Treat every token as an independent classification sub-task.

➤ Critical Challenges

- How do we handle tokens that do not belong to any of the classes?
- How do we handle entities that consist of multiple words?

Barack Obama Person the 44th President of the United States Title, was born in Honolulu, Hawaii Location. He graduated from Columbia University Org and Harvard Law School Org. In 2009 Date, Obama was elected as the first African American Ethnicity President of the United States Location. During his presidency, Obama implemented the Affordable Care Act Law and strengthened diplomatic relations with Cuba Location. He served two terms in office before being succeeded by President Donald Trump Title in 2017 Date.

Named Entity Recognition (NER)

➤ BIO tagging schema!

- B-(Beginning): Marks the first token of an entity
- I-(Inside): Marks subsequent tokens in the same multi-token entity.
- O (Outside): Marks tokens that do not belong to any entity.
 - **Example:** For the category **Person**, we use the tags **B-Person** and **I-Person**.

➤ Example:

- Barack -> B-Person
- Obama -> I-Person
- the -> B-Title
- 44th -> I-Title
- ...

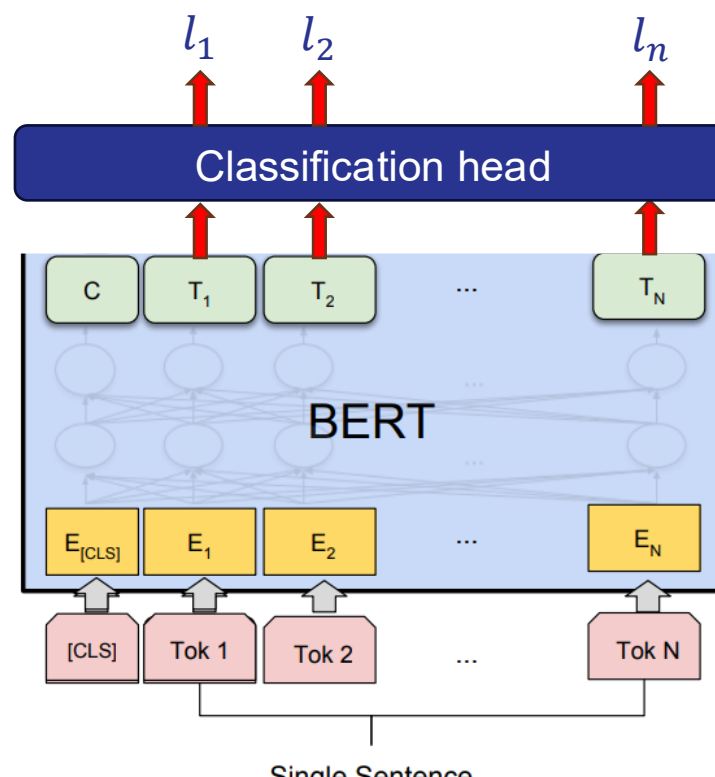
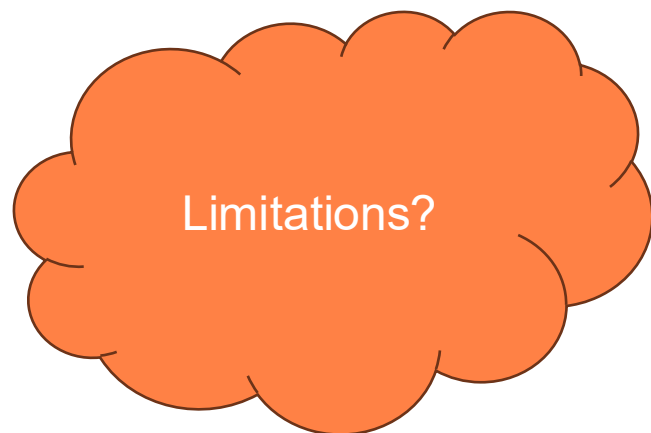
Barack Obama Person the 44th President of the United States Title, was born in Honolulu, Hawaii Location. He graduated from Columbia University Org and Harvard Law School Org. In 2009 Date, Obama was elected as the first African American Ethnicity President of the United States Location. During his presidency, Obama implemented the Affordable Care Act Law and strengthened diplomatic relations with Cuba Location. He served two terms in office before being succeeded by President Donald Trump Title in 2017 Date.

Named Entity Recognition (NER)

- SOTA as usual is achieved using transformers (encoder-only like BERT)
 - How we do this?

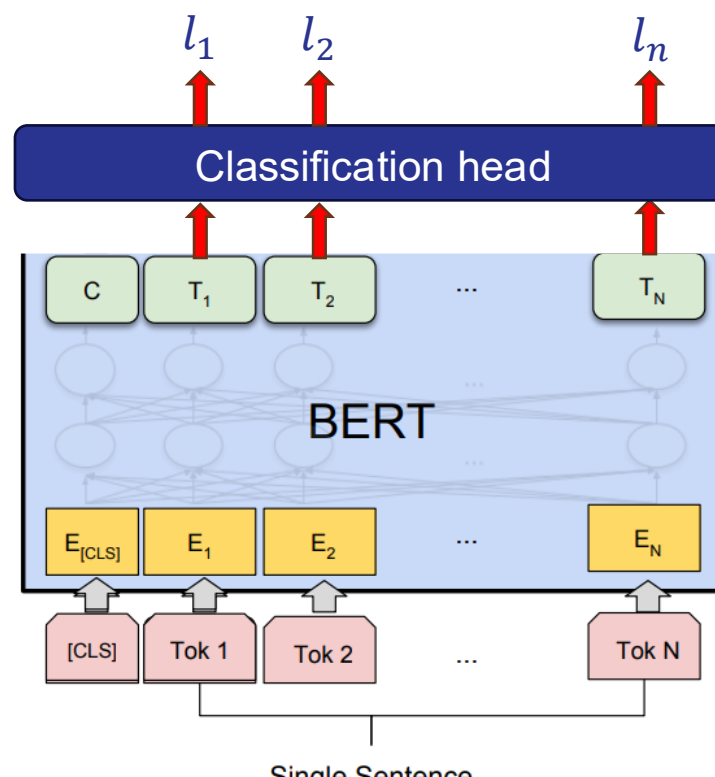
Named Entity Recognition (NER)

- SOTA as usual is achieved using transformers (encoder-only like BERT)
 - Unlike Sentiment Analysis (which uses only the [CLS] token), NER requires a **classification head on every token output**.
 - Each token's hidden state is mapped to a probability distribution over our labels (B, I, O categories).



Named Entity Recognition (NER)

- Does the model "know" the relationship between labels?
 - A Transformer might predict $l_1 = O$ and $l_2 = I\text{-}Person$ or $l_1 = B\text{-}Person$ and $l_2 = I\text{-}Date$, which are both "illegal" transitions according to the BIO schema



What if the model predicts:

$l_1 = O$
 $l_2 = I\text{-}Person$
 Or
 $l_1 = B\text{-}Person$
 $l_2 = I\text{-}Date$

Named Entity Recognition (NER)

➤ Solution:

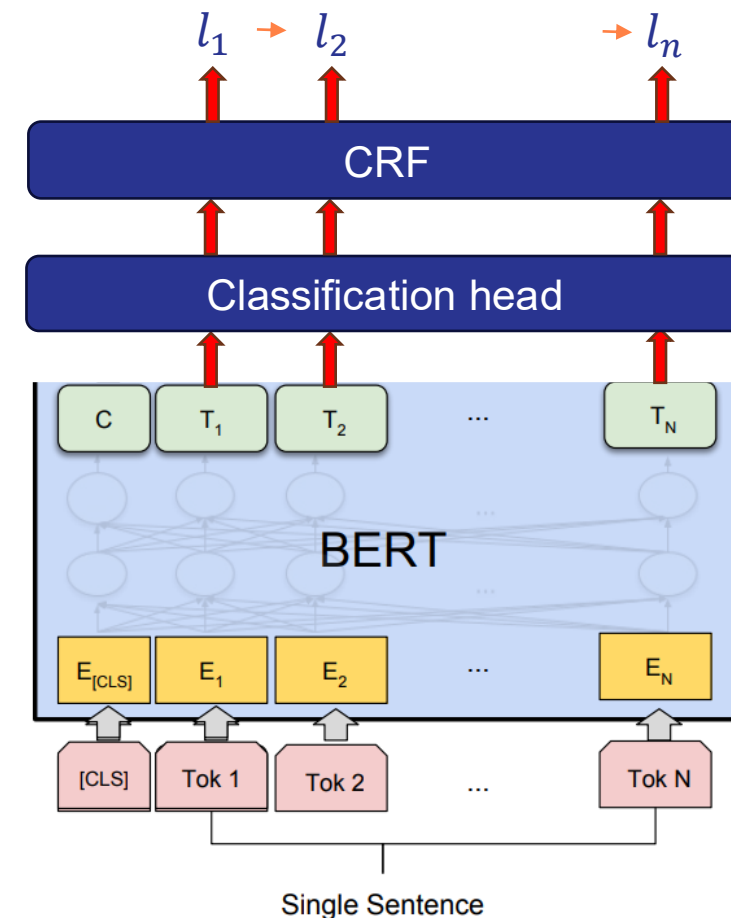
- add a Conditional Random Field (CRF) layer on top of the Transformer outputs to model the dependencies between labels.

➤ Conditional Random Field (CRF)

- Instead of making independent decisions for each token, the CRF looks at the **entire sequence** of labels.
- It learns a **Transition Matrix** between the labels.

$$P(y|x) = \frac{1}{Z(x)} e^{\sum_{t=1}^T (P(x) + A(y_{t-1}, y_t))}$$

BERT logits
Transition matrix



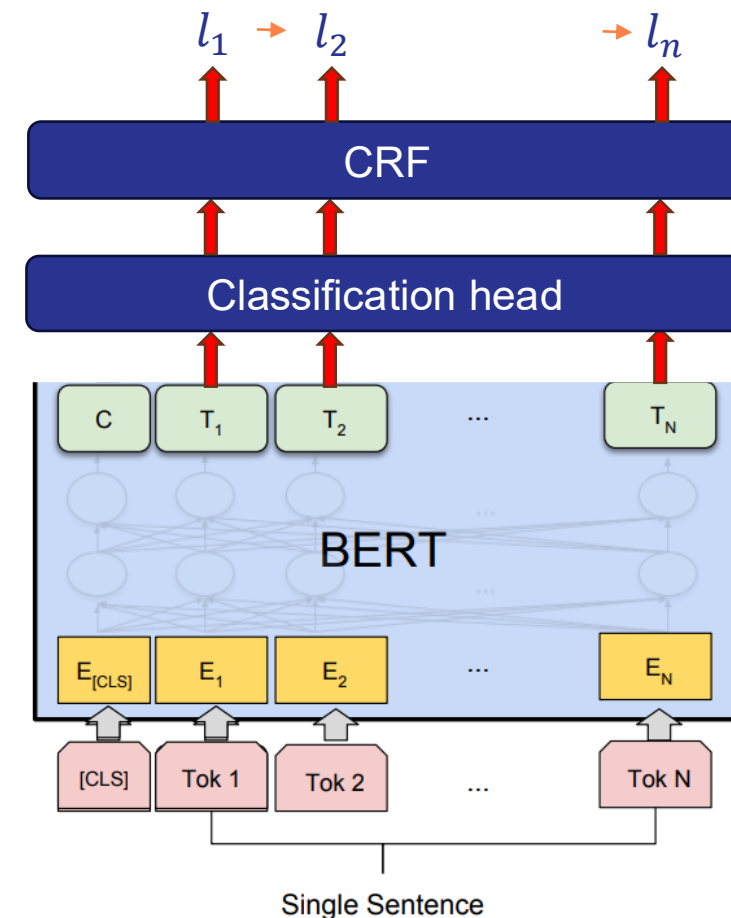
Named Entity Recognition (NER)

➤ Solution:

- add a Conditional Random Field (CRF) layer on top of the Transformer outputs to model the dependencies between labels.

➤ Conditional Random Field (CRF)

- Instead of making independent decisions for each token, the CRF looks at the **entire sequence** of labels.
- It learns a **Transition Matrix** between the labels.



$$P(y|x, y_{t-1}) = \frac{1}{Z(x)} e^{\sum_{t=1}^T (P(x) + A(y_{t-1}, y_t))}$$

BERT logits

Transition matrix

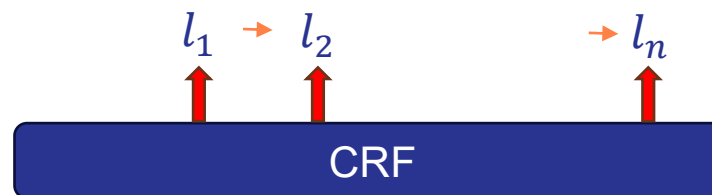
We can still do better!
Do you see any problem?

Named Entity Recognition (NER)

- Gap
 - In the BIO schema some transaction are impossible, namely $O \rightarrow I$ or $B-X \rightarrow I-Y$
- Why not make this a mathematical constraint on the transition matrix?
 - We can set the transition scores of impossible transitions in the matrix to $-\infty$.

Named Entity Recognition (NER)

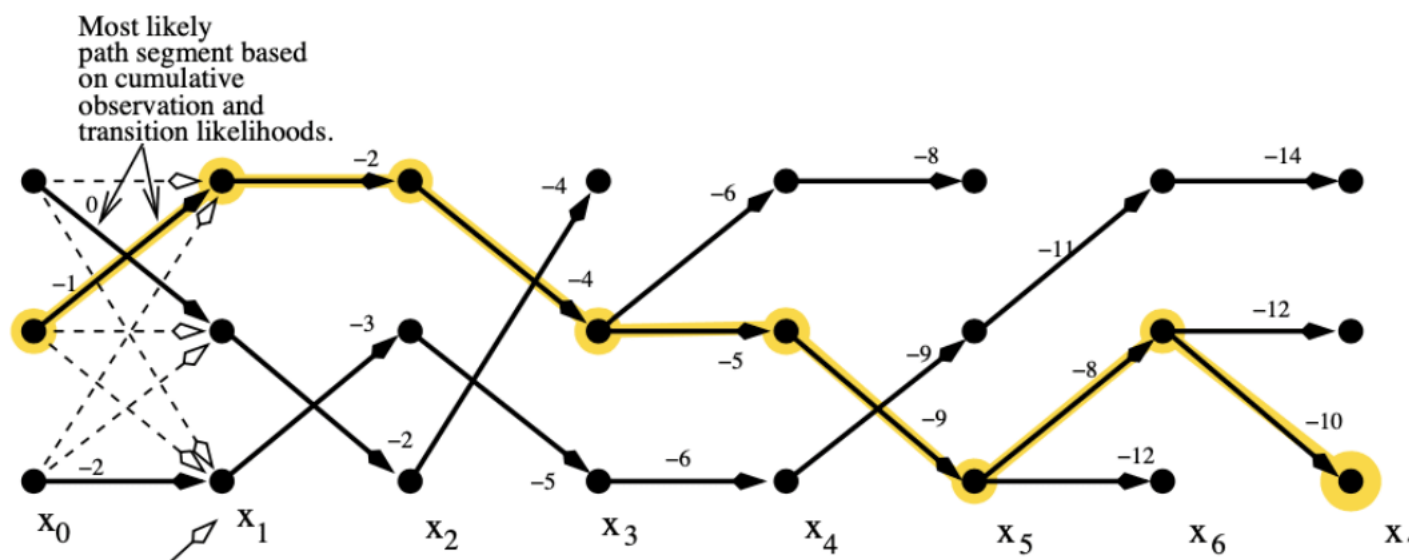
- How we select the tags now? (do inference)
 - Remember now we have a sequence!



- Greedy search? (picking the best label word-by-word)
- Brute force?
- Or any other text generation strategy?

Named Entity Recognition (NER)

- No! There is an efficient solution based on dynamic programming.
 - **Viterbi Decoding**
- Finds the absolute best global path efficiently in $O(TK^2)$ time.
 - Forward pass: As we move left-to-right, we don't lock in a single tag. For each possible tag at step t , we calculate and keep *only the highest-scoring path* that leads to it.
 - Backtracking: At the final token, we find the maximum total score, and follow our backpointers right-to-left to retrieve the optimal sequence.



Challenges with NLP tasks and BERT

- Batch optimization
- Long Context Windows.

Challenges: Batch optimization

➤ **Context:** Text data has high variance in sequence length. A single random batch might contain short sentences (e.g., 10 tokens) alongside long documents (e.g., 1024 tokens).

➤ **Problem:**

- Padding all sequences to match the absolute longest sequence in the batch forces the model to process thousands of useless [PAD] tokens.
- This drastically wastes GPU memory and computational power (since self-attention scales quadratically with sequence length).

Sent 1 (max length, 1024)

Sent 2 (512)

Pad to max

Sent 3 (512)

Pad to max

Sent 4 (400)

Pad to max

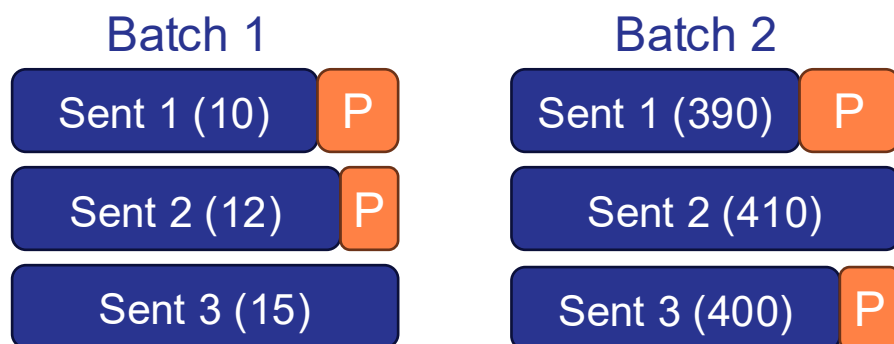
Sent 5 (600)

Pad to max

How to fix this?

Challenges: Batch optimization

- Instead of random sampling, group sentences of similar lengths into the same "buckets" before creating batches.
- **Dynamic Padding:** We now only pad sequences to match the longest sequence **within that specific batch**, rather than the absolute maximum of the entire dataset (e.g., 1024).
 - Drastically reduces the number of useless [PAD] tokens processed.
 - Significantly speeds up training



• `train_sampling_strategy` (`str`, *optional*, defaults to `"random"`) — The sampler to use for the training dataloader. The values are:

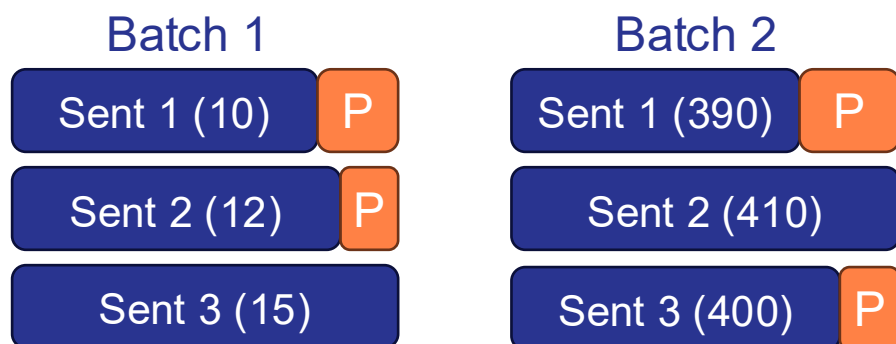
- `"random"`: Uses `RandomSampler` (default).
- `"sequential"`: Uses `SequentialSampler`.
- `"group_by_length"`: Uses `LengthGroupedSampler` to group samples of roughly the same length together to avoid padding and be more efficient).

Note: When using an `IterableDataset`, this argument is ignored.

- `length_column_name` (`str`, *optional*, defaults to `"length"`) — Column name for precomputed lengths. If the `grouping by length` will use these values rather than computing them on train startup. Ignored unless `train_sampling_strategy` is `"group_by_length"` and the dataset is an instance of `Dataset`.

Challenges: Batch optimization

- Instead of random sampling, group sentences of similar lengths into the same "buckets" before creating batches.
- **Dynamic Padding:** We now only pad sequences to match the longest sequence **within that specific batch**, rather than the absolute maximum of the entire dataset (e.g., 1024).
 - Drastically reduces the number of useless [PAD] tokens processed.
 - Significantly speeds up training



We are still wasting compute with padding.
How can we do better?

• `train_sampling_strategy` (`str`, *optional*, defaults to "random") — The sampler to use for the training dataloader. Values

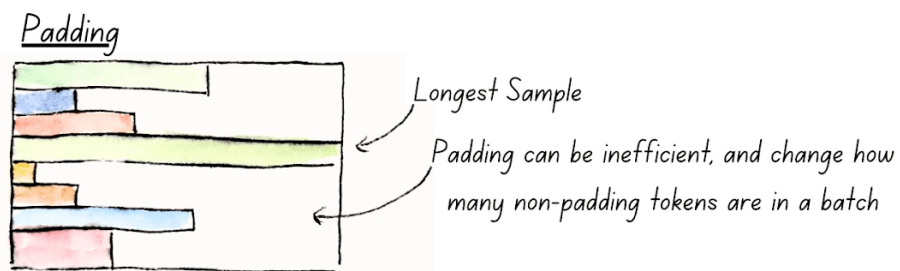
er to group samples of roughly the same length together.

Note: When using an `IterableDataset`, this argument is ignored.

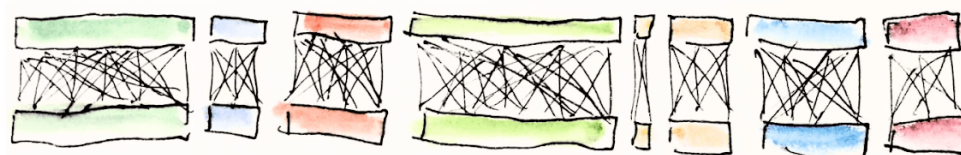
- `length_column_name` (`str`, *optional*, defaults to "length") — Column name for precomputed lengths. If the grouping by length will use these values rather than computing them on train startup. Ignored unless `train_sampling_strategy` is "group_by_length" and the dataset is an instance of `Dataset`.

Challenges: Batch optimization

- Unpadding and Sequence packing!
 - Instead of wasting space, we concatenate multiple short sequences together to form a single, dense sequence that perfectly fills up the model's maximum context window.
 - No more Padding tokens, no more waste of computation!



Attention masks ensure that samples are processed independently



Challenges: Long Context Windows

- Standard Transformer encoders (like BERT) have a strict maximum token limit (typically 512).

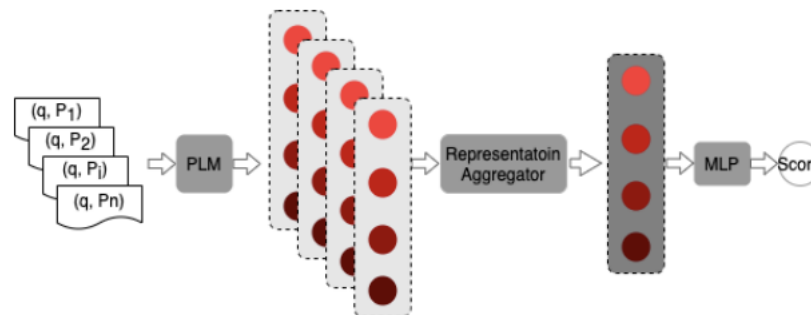


Challenges: Long Context Windows

- Standard Transformer encoders (like BERT) have a strict maximum token limit (typically 512).
- Strategy 1: Truncation
 - Simply cut the text to fit the limit.
 - **Head + Tail:** Instead of just taking the beginning, keeping the first X tokens and the last Y tokens often captures the most critical information in a document.

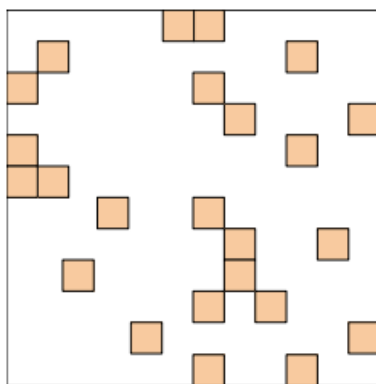
Challenges: Long Context Windows

- Standard Transformer encoders (like BERT) have a strict maximum token limit (typically 512).
- Strategy 2: Sliding Window (Chunking)
 - Break the long document into smaller, overlapping chunks (e.g., chunks of 512 tokens with a 50-token overlap).
 - Pass each chunk through the model individually, then aggregate the final outputs (e.g., average the predictions for classification or average of the intermediate representations).

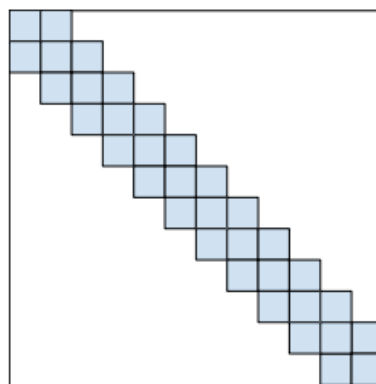


Challenges: Long Context Windows

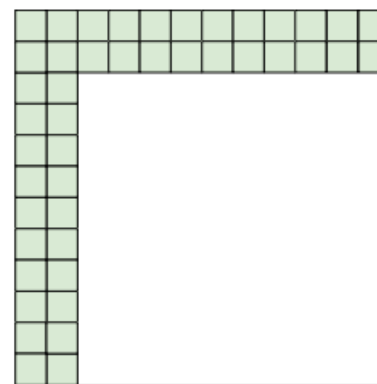
- Standard Transformer encoders (like BERT) have a strict maximum token limit (typically 512).
- Strategy 3: Specialized Architectures
 - Try a transformer architecture that supports large context like, LongFormer or BigBird, both use a form of sparse attention.



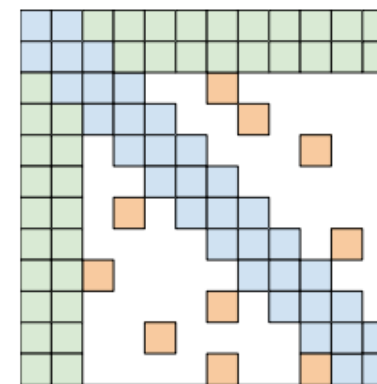
(a) Random attention



(b) Window attention



(c) Global Attention

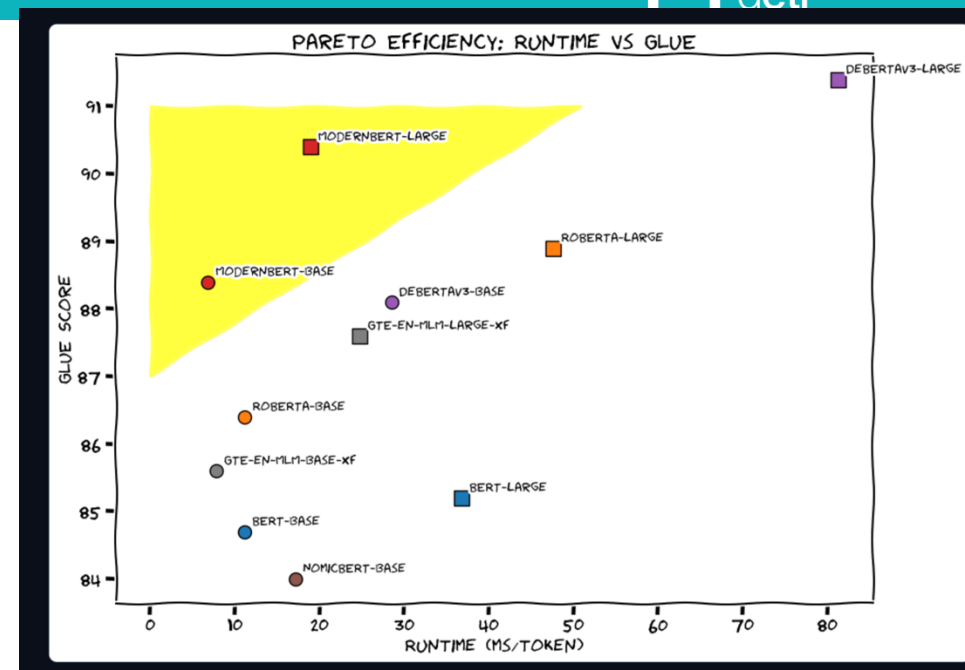


(d) BIGBIRD

ModernBERT (2024)

- Finally, the replacement of BERT!
 - Maximum sequence length: 8192 tokens
 - Better performance
 - Faster! (fast attentionV2, unpadding and sequence packing)

- Trained on 2 Trillion token dataset that includes code.
 - Uses 30% of masked tokens.



3. Assistant models (RLHF)

From GPT-3 to ChatGPT

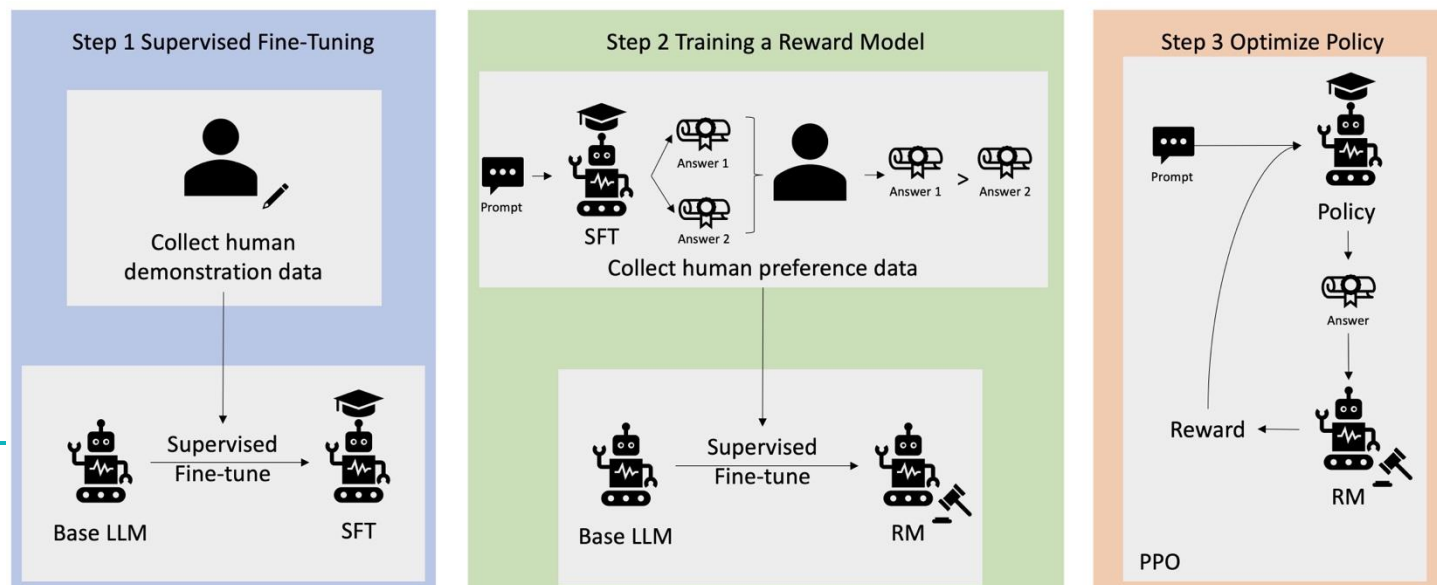
- How to go from a Language model to an assistant model?
- The base model (GPT-3): A pure "next-word predictor."
 - It acts like an advanced autocomplete. If you prompt it with "*How to bake a cake?*", it might just generate "*How to bake a pie?*" because it is mimicking a list of questions it saw on the internet.
- The chat model (chatGPT): An "instruction-following" model.
 - It understands the conversational format (User vs. Assistant) and knows its job is to *answer* the prompt, not just continue the text.

Model Alignment

- Alignment is how we steer a wild prediction engine into a safe product.
 - Shaping the model's behaviour to be:
 - Helpful (answers the prompt)
 - Honest (doesn't confidently lie)
 - Harmless (refuses dangerous or toxic requests)

➤ Reinforcement Learning from Human Feedback (RLHF) 2022

1. Supervised FT
2. Reward Modeling
3. Reinforcement Learning (PPO)



Model Alignment – Nowadays

- **Less is More (High-Quality SFT):**
 - We now know that a few thousand *perfectly* crafted human examples are better than millions of mediocre ones.
- **Don't use RL:** Traditional RLHF was mathematically unstable and required running multiple models at once.
- **Direct Preference Optimization (DPO):** The modern standard (used in Llama 3, etc.). It mathematically skips the "Reward Model" phase, directly updating the LLM's weights based on a dataset of "Chosen" vs. "Rejected" answers.

Model Alignment – Nowadays

➤ What about reasoning?

Model Alignment – Nowadays

➤ What about reasoning?

- Instead of training the model on [Question] -> [Answer], they train it on [Question] -> [Step 1] -> [Step 2] -> [Step 3] -> [Answer].
- Where the data comes from? Human annotation + Synthetic Generation

Perplexity and Uncertainty

- How "surprised" is the model by the text?
 - Standard metric for evaluating language models.
 - Measures the model's confidence in predicting the next token.
 - **Low Perplexity:** Highly predictable, standard phrasing.
 - **High Perplexity:** Unexpected, creative, or complex phrasing.
 - LLMs inherently favor low-perplexity paths.

$$\text{PPL}(X) = \exp \left\{ -\frac{1}{t} \sum_i^t \log p_{\theta}(x_i | x_{<i}) \right\}$$

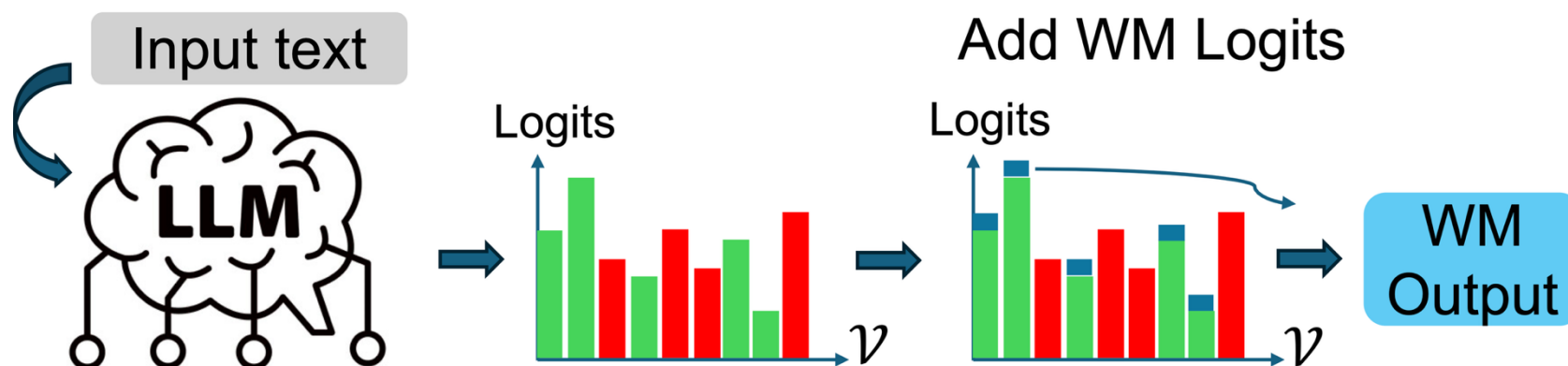
Formula of Perplexity from HuggingFace

The Hallucination Problem

- Why do LLMs confidently generate false information?
 - **Maximum Likelihood Flaw:** Training via MLE optimizes for *linguistic plausibility* (high probability sequences) rather than factual accuracy. The model prefers sounding correct over being correct.
 - **Exposure Bias:** Models are trained using Teacher Forcing (always seeing the true previous token). During autoregressive inference, generating a single suboptimal token cascades into larger, confident deviations.
 - **Parametric Memory:** Facts are stored implicitly as continuous weights rather than discrete database entries, leading to feature interference and blending of concepts.

Detecting AI Generated Text

- Current approach
 - **Statistical Classifiers:** Current detectors rely on proxy metrics like perplexity.
 - **Training Classifiers:** Use DL to train detectors.
 - **Logit Watermarking:** Modifying the generation step by pseudo-randomly splitting the vocabulary and artificially boosting the logits of a "Green List" of tokens for robust, cryptographic detection.



Future directions?

- More efficient LLMs?
 - **MAMBA** (state space model)
- Or completely ignore the LLMs?
 - JEPA (Joint-Embedding Predictive Architecture)





15 minutes

Hands-On Lab

Thank You!

See you next week!

Lab submission due in 1 week
